Theory of Algorithms – Transform and Conquer

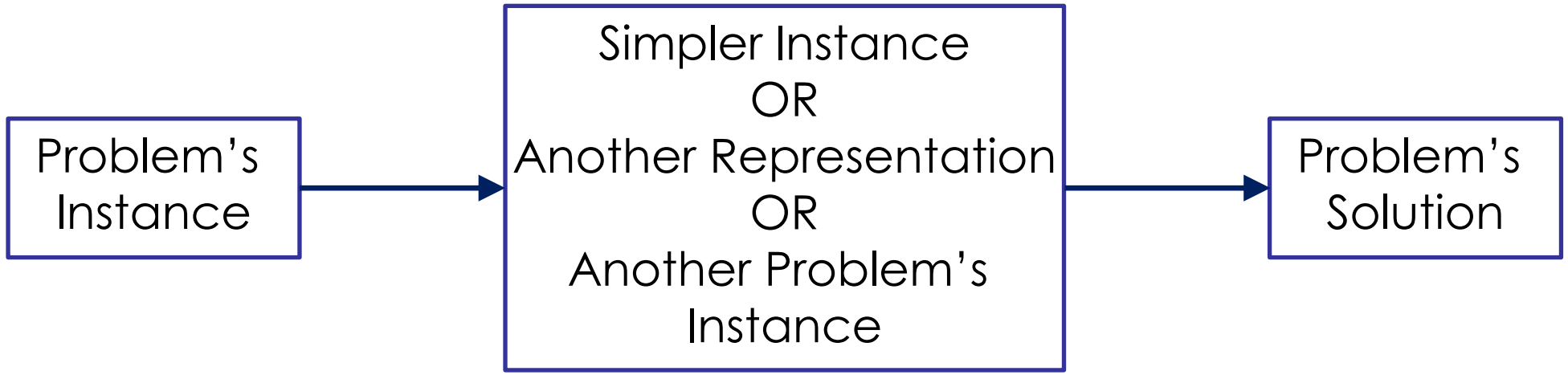
Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





Introduction

- "The Secret of Life … to replace one worry with another" – Charles M. Schultz
- Transform the problem in some way and solve the transformed problem



Approaches

- Instance Simplification = a more convenient instance of the same problem
  - Presorting
  - Gaussian elimination
- Representation Change = a different representation of the same instance
  - Balanced search trees (CS2)
  - Heaps and heapsort (CS2)
  - Polynomial evaluation by Horner's rule
  - Binary exponentiation
- Problem Reduction = a different problem altogether
  - Lowest Common Multiple
  - Reductions to graph problem

Transform and Conquer

- Instance Simplification
  - Gaussian elimination
  - Presorting
- Representation Change
  - Balanced search trees (CS2)
  - Heaps and heapsort (CS2)
  - Polynomial evaluation by Horner's rule
  - Binary exponentiation
- Problem Reduction
  - Lowest Common Multiple
  - Reductions to graph problem

Instance Simplification: Gaussian Elimination

- Solution of a fully-determined system of linear equations
- First transform to upper triangular form by Gaussian elimination (simplification) then solve by backward substitution (conquer)
- $$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ 0 & b_{22} & b_{23} & b_{24} \\ 0 & 0 & b_{33} & b_{34} \\ 0 & 0 & 0 & b_{44} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}$$
- Forward Elimination is $\Theta(n^3)$ while backward substitution is $\Theta(n^2)$

Instance Simplification: Presorting

- Solve instance of problem by preprocessing the problem to transform it into another simpler/easier instance of the same problem
- Many problems involving lists are easier when list is sorted:
  - Searching
  - Computing the median (selection problem)
  - Finding repeated elements
  - Convex Hull and Closest Pair
- Efficiency:
  - Introduce the overhead of an $\Theta(n \log n)$ preprocess
  - But the sorted problem often improves by at least one base efficiency class over the unsorted problem (e.g., $\Theta(n^2) \rightarrow \Theta(n)$)

Is Presorting Better?

- Is transformation by sorting better than brute force for these methods?
  - Closest pair
  - Checking uniqueness
  - Finding the min and max
  - Finding the median
  - Searching for key in unordered array

Is Presorting Better?

- Is transformation by sorting better than brute force?
- Sorting is Θ ($n \log n$) so transformation to sorting is only worthwhile if other algorithms are less efficient
  - Closest pair – $\Theta(n^2) \rightarrow \Theta$ (n log n), so presort
  - Checking uniqueness – Brute force is Θ (n^2), so presort
  - Finding the min and max – Brute force is Θ (n), so NO presort
  - Finding the median – Brute force is Θ (n^2), so presort
  - Searching unordered array – Brute force is Θ (n), so NO presort

Example: Presorted Selection

- Find the kth smallest element in A[1],…, A[*n*]
- Special cases:
  - Minimum: *k* = 1, Maximum: *k* = *n*, Median: *k* = $\lceil n/2 \rceil$
- Presorting-based algorithm:

```
Sort list
Return A[k]
```

- Partition-based algorithm
  (Variable Decrease & Conquer):

```
Pivot at A[s] using Partitioning from Quicksort
if s=k return A[s]
else if s<k repeat with sublist A[s+1],..., A[n]
     else if s>k repeat with sublist A[1],..., A[s-1]
```

Notes on the Selection Problem

- Presorting-based algorithm:
  - $\Theta(n \log n) + \Theta(1) = \Theta(n \log n)$
  - Also identifies the k smallest elements (not just the kth)
- Partition-based algorithm (Variable decrease & conquer):
  - Worst case: $\Theta(n^2)$
  - Best case: $\Theta(n)$
  - Average case: $\Theta(n)$
- Simpler linear (brute force) algorithm is better in the case of max & min

Transform and Conquer

- Instance Simplification
  - Presorting
  - Gaussian elimination
- Representation Change
  - Balanced search trees
  - Heaps and heapsort
  - Polynomial evaluation by Horner's rule
  - Binary exponentiation
- Problem Reduction
  - Lowest Common Multiple
  - Reductions to graph problem

Evaluating Polynomials

- Polynomial of form

$$p(x) = a_n x^n + \cdots + a_i x^i + \cdots + a_0$$

- Example:
  - $p(x) = 2x^4 - x^3 + 3x^2 + x - 5$
  - Evaluate for $x = 3$
- The traditional, obvious, brute force approach:
  - $p(3) = 2(3)^4 - 3^3 + 3(3)^2 + 3 - 5$

Brute Force Polynomial

- For a polynomial of size n, just the first term $a_n x^n$ requires n multiplications using brute force
- We can improve on this by efficiently calculating x^n
- But Horner's rule does even better for large polynomials
  - and it's very easy

Horner's Rule

- Factor x out as much as possible
- Example:
  - $p(x) = 2x^4 - x^3 + 3x^2 + x - 5$
  - $= (2x^3 - x^2 + 3x + 1)\,x - 5$
  - $= ((2x^2 - x + 3)\,x + 1)\,x - 5$
  - $= (((2x - 1)\,x + 3)\,x + 1)\,x - 5$
- How does this help?

Horner's Rule vs Brute Force

- Solve: $p(x) = 2x^3 - x^2 - 6x + 5$ for $x = 3$
- By Brute Force:
  - 2*3*3*3-3*3-6*3+5=32
  - 5 mult, 3 add/sub
- By Horner's Rule: $p(x) = ((2x - 1)\,x - 6)\,x + 5$
  - C[]: 2 -1 -6 5
  - p: 2 2*3-1=5 5*3-6=9 9*3+5=32
  - 3 mult, 3 add/sub

Horner's Rule Pseudocode

```
double horner(coefficients[0..n], x):
    p = coefficients[n]
    for i = n - 1 downto 0:
        p = x * p + coefficients[i]
    return p
```

- Can you think of a polynomial where Horner's Rule is no help at all?

Horner's Rule Efficiency

- Basic operations are multiplication and addition
  - Let number of multiplications = M(n)
  - Let number of additions = A(n)
  - $M(n) = A(n) = \sum_{i=0}^{n-1} 1 = n$

```
double horner(coefficients[0..n], x):
    p = coefficients[n]
    for i = n - 1 downto 0:
        p = x * p + coefficients[i]
    return p
```

- For the entire polynomial it takes as many multiplications as the Brute Force method uses for its first term
  - Brute force needs n + (n-1) + … + 1 = $\Theta(n^2)$ for entire (complete) polynomial.

Binary Exponentiation

- Problem: solve a^n efficiently
  - Solution uses the binary representation of n
- Represent n as a polynomial p(x), with x = 2
  - $n = p(x) = b_k x^k + \cdots + b_i x^i + \cdots + b_0$
  - Then coefficients of polynomial $(b_k \ldots b_i \ldots b_0)$ are digits of the binary presentation
- Example:
  - $n = 13, p(2) = 1 * 2^3 + 1 * 2^2 + 0 * 2 + 1$
  - Binary 13 is 1 1 0 1

$a^n = a^{p(2)}$

Horner's Rule for p(2)

```
p=1   // leading digit is
      // always 1
for i = k-1 downto 0:
      p = 2 * p + C[i]
```

Implications for $a^n = a^{p(2)}$

$a^p = a^1$

```
for i = k-1 downto 0:
      a^p = a^(2 * p + C[i])
```

$$a^{2p+C[i]} = a^{2p} \times a^{C[i]} = (a^p)^2 \times a^{C[i]} = \begin{cases} (a^p)^2 & if\ C[i] = 0 \\ (a^p)^2 \times a & if\ C[i] = 1 \end{cases}$$

```
prod = prod*prod
if C[i] == 1:
   prod = prod*a
```

Horner's Rule for p(x)

```
p=C[n]
for i = n-1 downto 0:
     p = x*p + C[i]
return p
```

Horner's Rule for a^n

```
p=a
for i = k-1 downto 0:
     p = p*p
     if C[i] == 1:
          p = p*a
return p
```

Horner Exponentiation: example

- Solve a^n for n = 13 exploiting Horner's Rule
  - $p(x) = 1x^3 + 1x^2 + 0x + 1 = \big((x+1)x + 0\big)x + 1$ at $x = 2$
  - C[]: 1 1 0 1
  - p: 1 1*2+1=3 3*2+0=6 6*2+1=13
  - a^p: a^1 $(a^1)^2 * a^1$ $(a^3)^2 * a^0$ $(a^6)^2 * a^1 = a^{13}$
- Since $a^{n*2+d} = (a^n)^2 * a^d$ and d is only 0 or 1

Efficiency of Horner Exponentiation

- Basic operation is multiplication, M(n)
- Brute force exponentiation:
  - a^n = a*a…*a (n-1 times)
  - M(n)=n-1 ∈ Θ (*n*)
- Horner exponentiation:
  - At most 2 multiplications on each loop iteration
  - k = length of bit representation of n, so $k-1 = \lfloor log_2 n \rfloor$
  - $k - 1 \leq M(n) \leq 2(k-1)$
  - $\Longrightarrow \mathrm{M(n)} \leq 2\ \lfloor log_2 n \rfloor$ ∈ Θ (log *n*)

Transform and Conquer

- Instance Simplification
  - Presorting
  - Gaussian elimination
- Representation Change
  - Balanced search trees
  - Heaps and heapsort
  - Polynomial evaluation by Horner's rule
  - Binary exponentiation
- Problem Reduction
  - Lowest Common Multiple
  - Reductions to graph problem

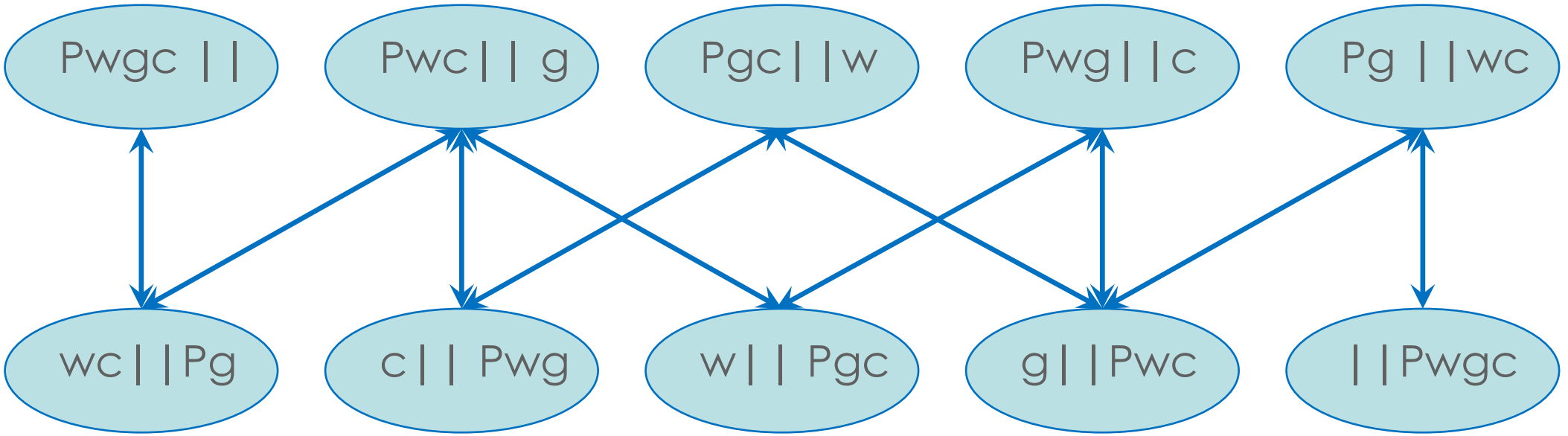
Problem Reduction

- Used in Complexity Theory to classify problems
- Computing the Least Common Multiple:
  - The LCM of two positive integers *m* and *n* is the smallest integer divisible by both *m* and *n*
  - Problem Reduction: LCM(*m*, *n*) = *m* * *n* / GCD(*m*, *n*) Example: LCM(24, 60) = 1440 / 12 = 120
- Reduction of Optimization Problems:
  - Maximization problems seek to find a function's maximum. Conversely, minimization seeks to find the minimum
  - Can reduce between: min *f(x)* = - max [- *f(x)*]

Reduction to Graph Problems

- Reduce a problem to a graph-based solution
  - Typically find solution path from one node to another
- State-Space Graphs:
  - Vertices represent states and edges represent valid transitions between states
  - Start and goal vertices
  - Widely used in AI
- Example:
  - River Crossing Puzzle [(P)easant, (w)olf, (g)oat, (c)abbage]
  - Goat&wolf, and goat&cabbage cannot be left alone
  - Get everyone across the river – boat caries peasant + one more item
  - Find path between given initial/end state; want minimal 'length' path

River Crossing Graph



Some Strengths and Weaknesses of Transform and Conquer

- Strengths:
  - Allows powerful data structures to be applied (via representation change e.gtrees)
  - Effective in Complexity Theory
- Weaknesses:
  - Can be difficult to derive (especially reduction)